EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Application-Controlled Agent Workflows

Instructor:
Arash Saifhashemi

EE 599 : AGENTIC PROGRAMMING

Application-Controlled Agent Workflows

Session 04 : Part I of II : Fixed, step-by-step, and planner-based control

CENTRAL IDEA

Models propose. Application code validates, executes, and decides.

Who controls action selection, execution, and acceptance?

One A100 recommendation, no provisioning

INPUT

```
Recommend one A100 in us-west-2 tonight if there is capacity
and we have quota.
```

CAPACITY EVIDENCE

One `CapacityFinding` for this request.

JOIN

QUOTA EVIDENCE

One `QuotaFinding` for the same request.

CONSTRAINT

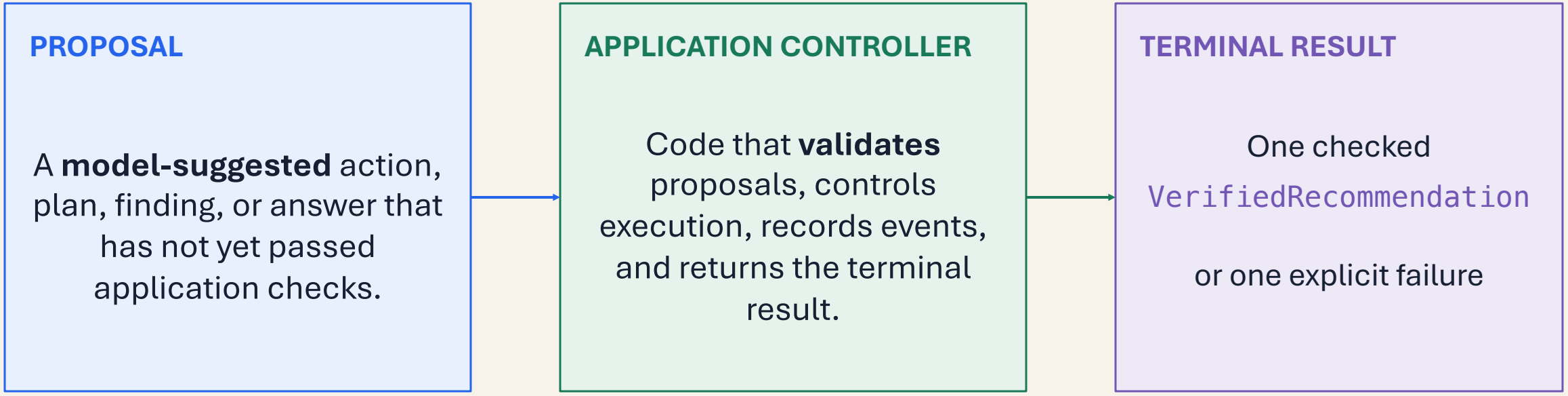
Produce a recommendation. Do not allocate a GPU.

EXPECTED OUTPUT

`VerifiedRecommendation` or `ExplicitFailure`

Fixed data: this example uses hypothetical course data, not live cloud inventory.

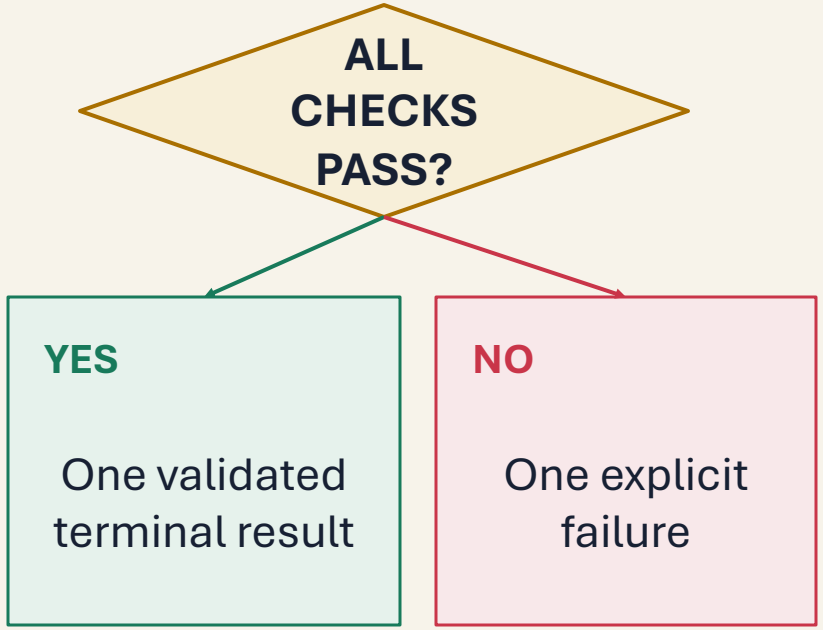
The application owns the decision boundary



Success requires evidence and one checked terminal result

THE APPLICATION ACCEPTS ONLY WHEN

1. The request and finding IDs match.
2. Both required findings exist.
3. Capacity and quota satisfy the task constraints.
4. No provisioning or other unauthorized mutation occurs.
5. The application records the run and returns exactly one terminal result.



Failure worth avoiding: a plausible recommendation based on incomplete evidence.

Questions we will answer

1. Why is a model's **answer** not yet a **decision**?
2. What **evidence** and **checks** must pass before **acceptance**?
3. Who **proposes**, **executes**, **validates**, and returns the result?
4. When should control be **fixed**, **step-by-step**, or **planner-based**?
5. How do we validate both a **planner's** plan and its **execution**?

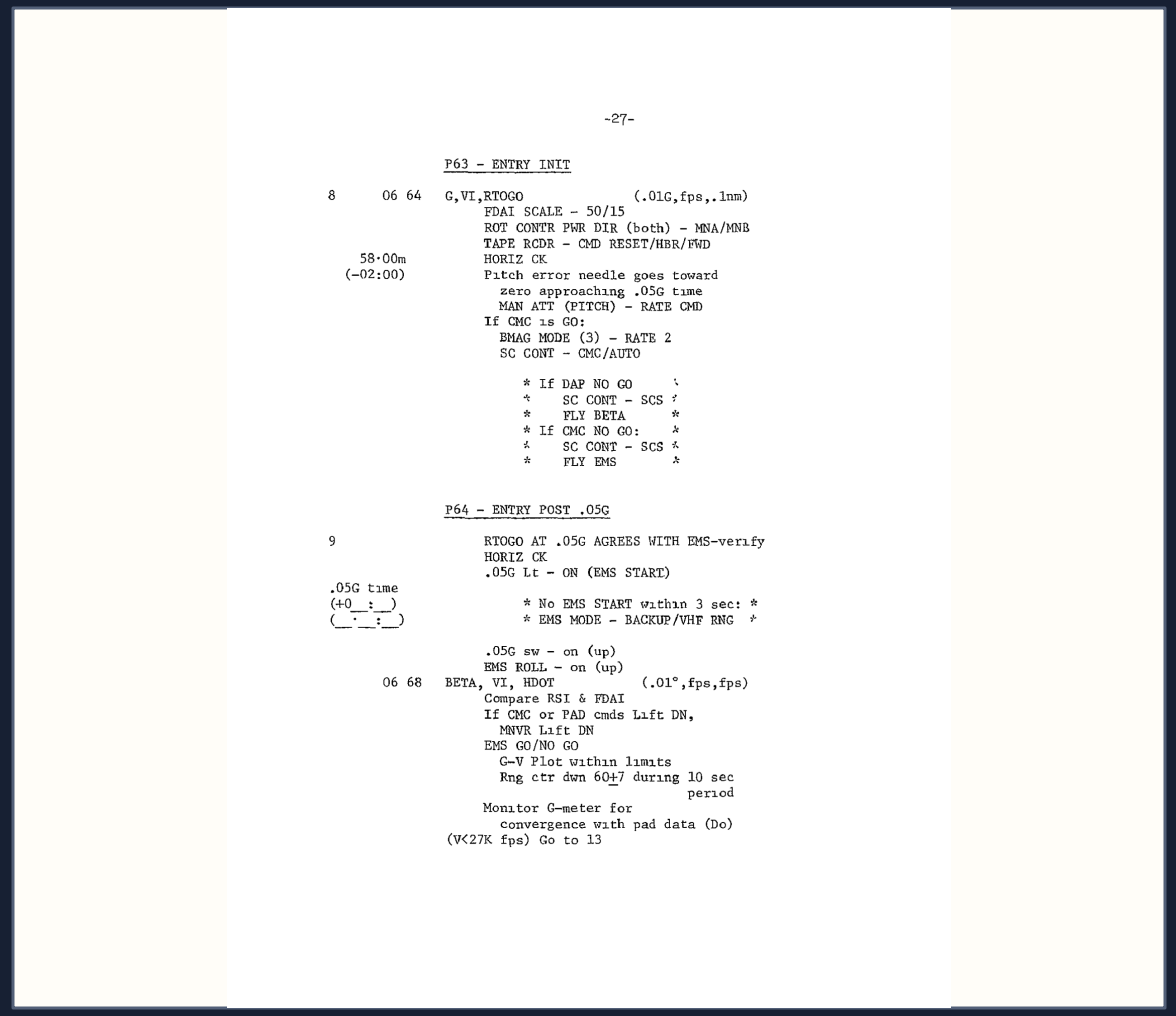
SECTION 2

Application-owned specifications and exact checks

How does application code turn policy into observable acceptance and failure conditions?

TECHNICAL MAPPING

An enforceable check must be explicit enough to inspect, execute, and record.



-27-

P63 - ENTRY INIT

8 06 64 G,VI,RTOGO (.01G,fps,.1nm)
FDAI SCALE - 50/15
ROT CONTR PWR DIR (both) - MNA/MNB
TAPE RCDR - CMD RESET/HBR/FWD
58:00m HORIZ CK
(-02:00) Pitch error needle goes toward
zero approaching .05G time
MAN ATT (PITCH) - RATE CMD
If CMC is GO:
BMAG MODE (3) - RATE 2
SC CONT - CMC/AUTO

* If DAP NO GO *
* SC CONT - SCS *
* FLY BETA *
* If CMC NO GO: *
* SC CONT - SCS *
* FLY EMS *

P64 - ENTRY POST .05G

9 RTOGO AT .05G AGREES WITH EMS-verify
HORIZ CK
.05G Lt - ON (EMS START)
.05G time
(+0__:__) * No EMS START within 3 sec: *
(__:__:__) * EMS MODE - BACKUP/VHF RNG *

.05G sw - on (up)
EMS ROLL - on (up)
06 68 BETA, VI, HDOT (.01°,fps,fps)
Compare RSI & FDAI
If CMC or PAD cmds Lift DN,
MNVR Lift DN
EMS GO/NO GO
G-V Plot within limits
Rng ctr dwn 60±7 during 10 sec
period
Monitor G-meter for
convergence with pad data (Do)
(V<27K fps) Go to 13

NASA, Apollo Entry Summary Document, 1969

Running example: recommend one A100 without provisioning

USER REQUEST

Recommend one A100 in us-west-2 tonight if there is capacity and we have quota.

APPLICATION TASK SCOPE

Produce a recommendation.
Do not allocate a GPU.

SCENARIO INPUTS

- `requestId = "req-599-04"`
- `region = "us-west-2"`
- `timeWindow = "tonight"`

REQUIRED EVIDENCE

`CapacityFinding`
`QuotaFinding`

REQUEST	CAPACITY	QUOTA	CANDIDATE	DECISION
req-599-04	missing	missing	not started	pending

Acceptance criteria for the A100 recommendation

First predict: what additional checks must pass before this answer can reach the user?

01 CAPACITY
At least **one** A100 is available in us-west-2 for the requested window.

02 QUOTA
The department has **quota** for at least one A100.

Acceptance criteria for the A100 recommendation

First predict: what additional checks must pass before this answer can reach the user?

01 CAPACITY
At least one A100 is available in us-west-2 for the requested window.

02 QUOTA
The department has quota for at least one A100.

03 SAME REQUEST
Findings and candidate share `requestId` = "req-599-04".

04 NO SIDE EFFECT
`ExecutionTrace` contains no executed `provisionGpu` action.

Acceptance criteria for the A100 recommendation

The application accepts the answer only when all six observable conditions pass.

01 CAPACITY
At least one A100 is available in us-west-2 for the requested window.

02 QUOTA
The department has quota for at least one A100.

03 SAME REQUEST
Findings and candidate share `requestId` = "req-599-04".

04 NO SIDE EFFECT
`ExecutionTrace` contains no executed `provisionGpu` action.

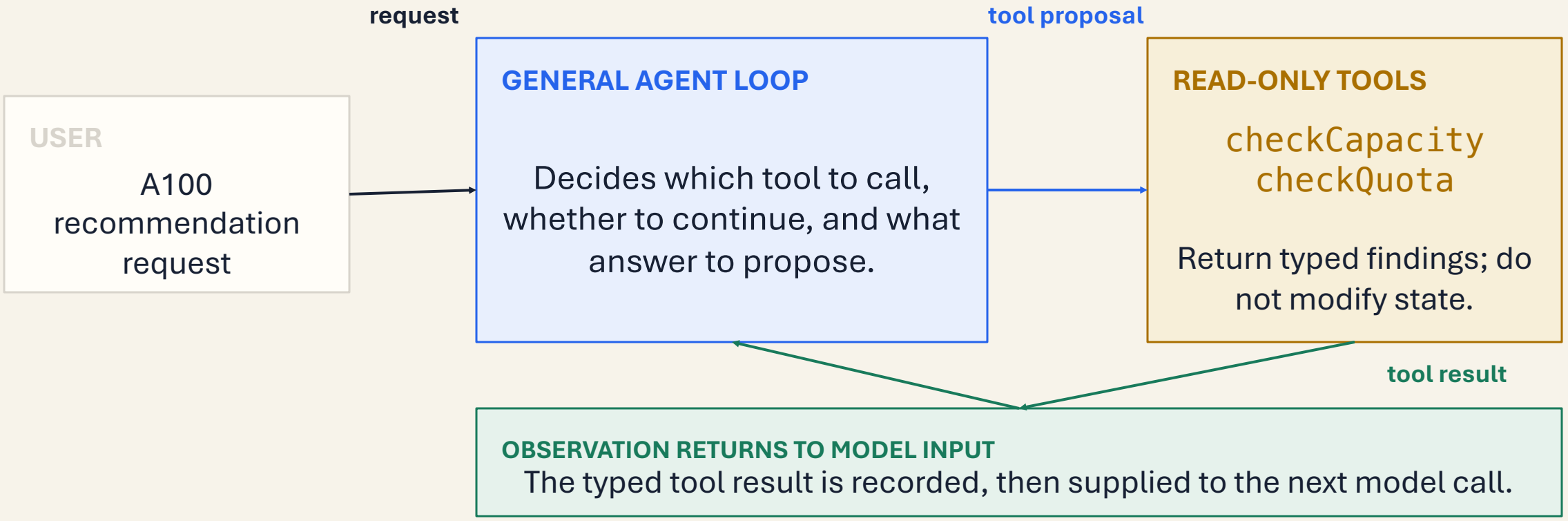
05 EVIDENCE CITATION
Candidate cites the exact `CapacityFinding` and `QuotaFinding`.

06 RETURN ONE OUTCOME
- All six checks pass: return a VerifiedRecommendation
- Any check fails: return an ExplicitFailure

The controller returns exactly one verified outcome. Never an unverified or partial recommendation.

Baseline: a model-controlled step-by-step loop

After each tool result, the model proposes the next action or stops. This is a ReAct-style loop.



A model-controlled stopping decision can end the loop before application-required evidence is present.

Would you accept this recommendation?

USER REQUEST

Recommend one A100 in us-west-2 tonight if there is capacity and we have quota.

APPLICATION-OWNED AGENT TASK SPECIFICATION

Obtain both `CapacityFinding` and `QuotaFinding` before recommending deployment.

CapacityFinding confirms that at least one A100 is available.

CANDIDATE RECOMMENDATION

“Use one available A100 in us-west-2 tonight.”

Should we return this?

Missing quota evidence invalidates the recommendation

USER REQUEST

Recommend one A100 in us-west-2 tonight if there is capacity and we have quota.

APPLICATION-OWNED AGENT TASK SPECIFICATION

Obtain both `CapacityFinding` and `QuotaFinding` before recommending deployment.

CANDIDATE RECOMMENDATION

“Use one available A100 in us-west-2 tonight.”

CAPACITY EVIDENCE PRESENT

`CapacityFinding` reports available = 4

REQUIRED `QuotaFinding` IS MISSING

No quota finding was recorded for this request.

REQUEST	CAPACITY	QUOTA	CANDIDATE	DECISION
ready	present	missing	rejected	`ExplicitFailure`

Missing quota evidence

Unvalidated model output

```
{
  "requestId": "req-599-04",
  "answer": "Use an A100 in us-west-2.",
  "capacityFindingId": "cap-usw2-0913",
  "quotaFindingId": null
}
```

1 : PARSE THE RECORD

Pydantic rejects null for a required string.

2 : RETRIEVE AND MATCH FINDINGS

Retrieve both findings; each `requestId` must equal req-599-04.

3 : CHECK THE CONTENTS

Capacity and quota must meet the task requirements.

The recommendation sounds plausible, but it cannot be validated: quotaFindingId is null, so the required QuotaFinding cannot be retrieved.

Core components

The user sends one request; the application controller validates the work and returns the final response.

USER OR CLIENT

Sends `DeploymentRequest`; receives one terminal response.

APPLICATION CONTROLLER

Validates proposals, records events, and returns the final response.

Core components

Add workflow selection and the application code that runs the selected sequence.

USER OR CLIENT

Sends `DeploymentRequest`; receives one terminal response.

APPLICATION CONTROLLER

Validates proposals, records events, and returns the final response.

NAVIGATOR MODEL CALL (ROUTER)

Proposes one allowed `WorkflowSelection`; does not start it.

WORKFLOW

Orders model, agent, tool, and verification steps.

Core components

The complete system also uses narrow evidence workers and a separate verification stage.

USER OR CLIENT
Sends `DeploymentRequest`; receives one terminal response.

APPLICATION CONTROLLER
Validates proposals, records events, and returns the final response.

NAVIGATOR MODEL CALL (ROUTER)
Proposes one allowed `WorkflowSelection`; does not start it.

WORKFLOW
Orders model, agent, tool, and verification steps.

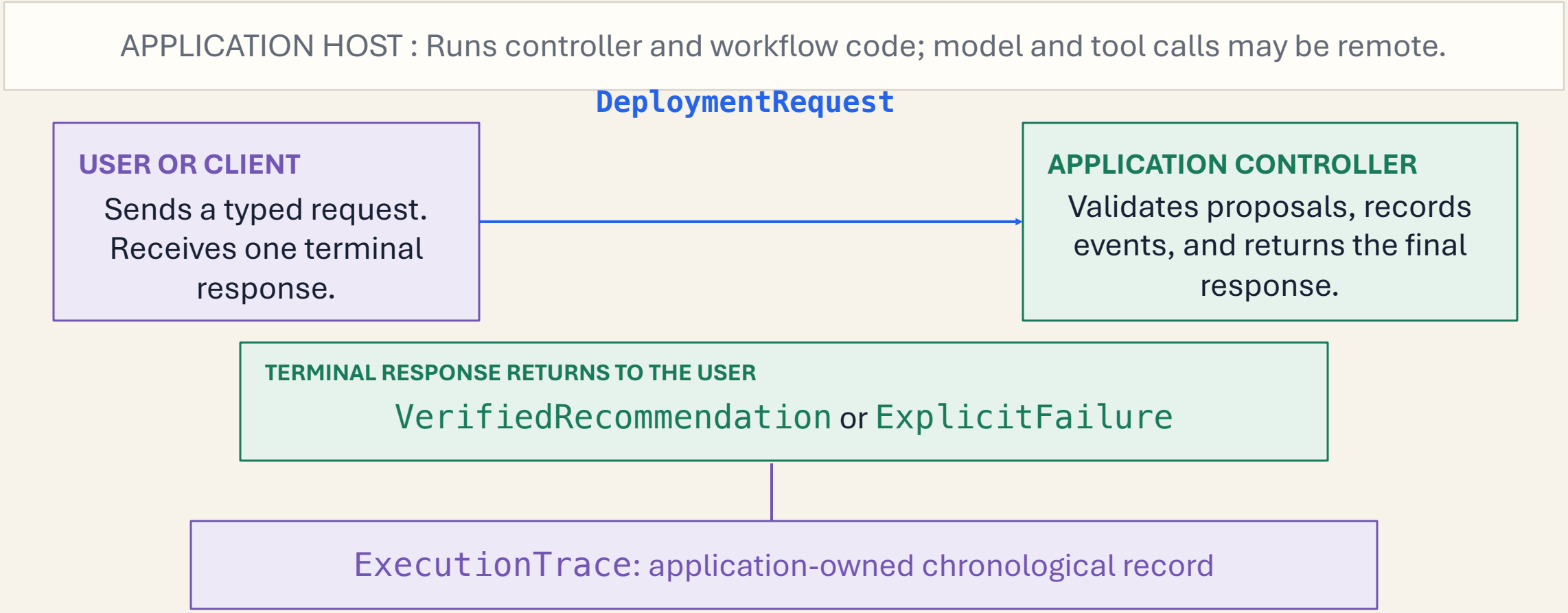
SPECIALIST AGENTS + TOOLS
Run narrow evidence tasks with approved tools.

VERIFICATION STAGE
Runs exact checks; the controller decides acceptance.

> Models and specialist agents propose or generate results; application code owns control and acceptance.

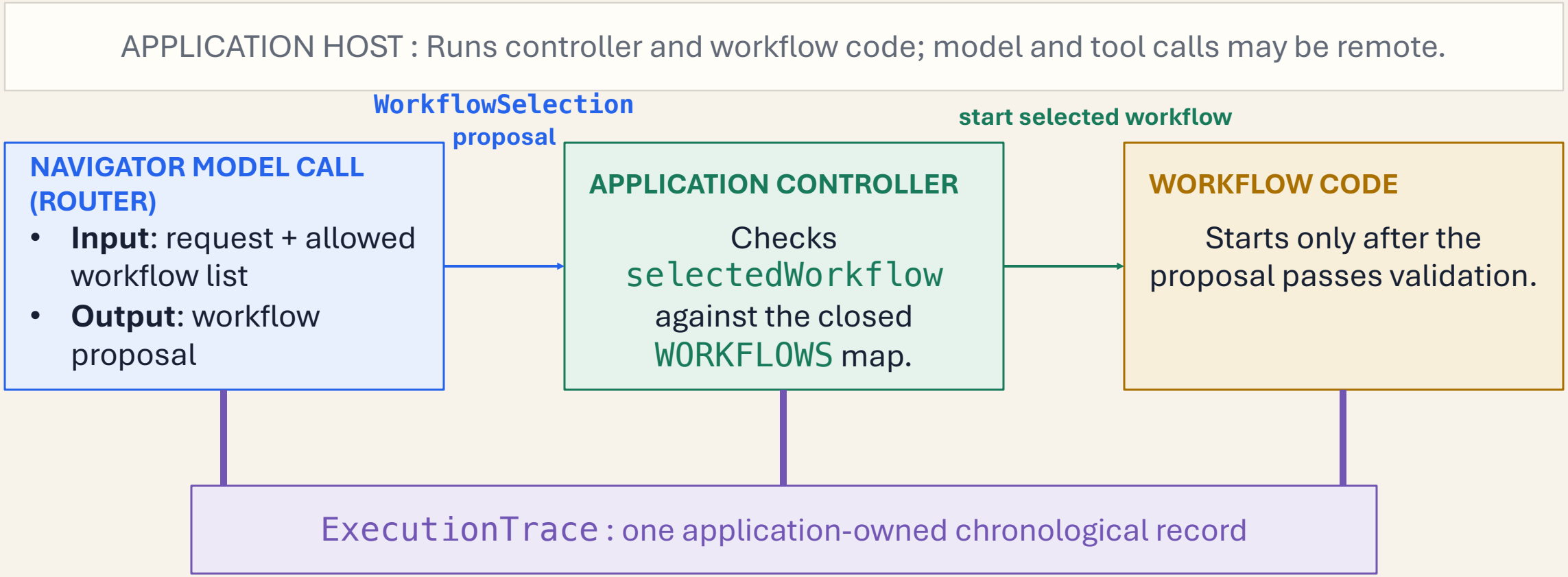
A100 recommendation architecture

The controller receives a typed request and returns exactly one terminal response.



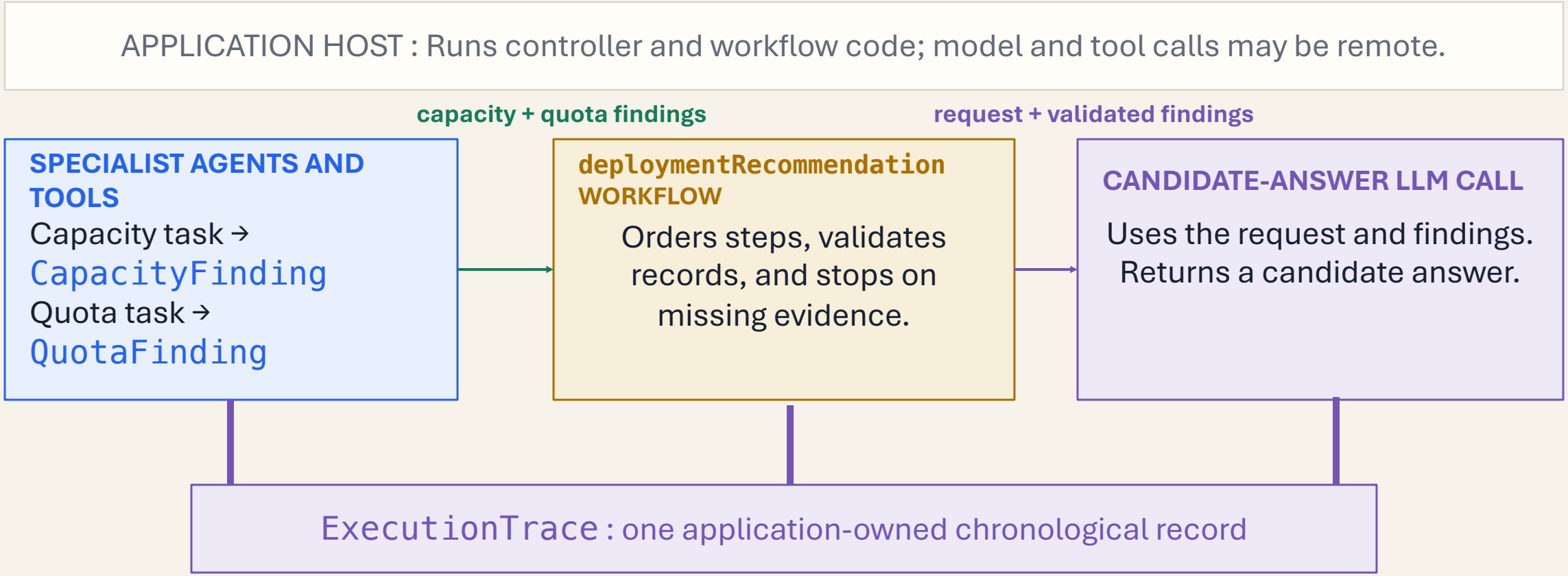
A100 recommendation architecture

The **Navigator** proposes a name; application code validates it before workflow code starts.



A100 recommendation architecture

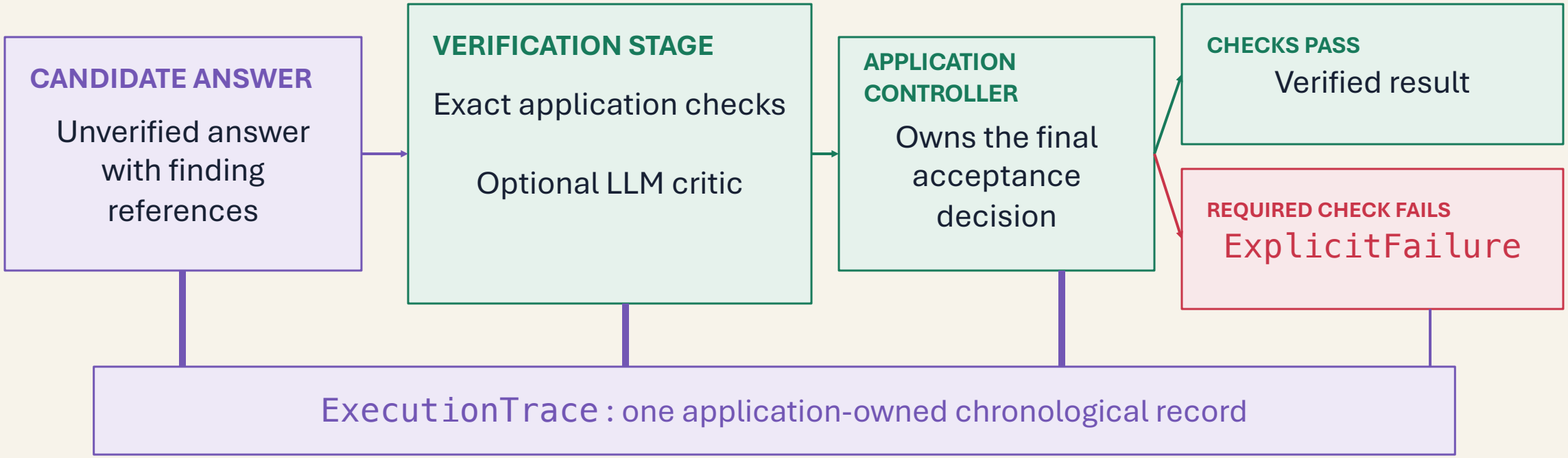
Specialists, tools, and an LLM call run under application control.



A100 recommendation architecture

Exact checks run before the controller chooses the **user response**.

APPLICATION HOST : Runs controller and workflow code; model and tool calls may be remote.



From user request to one checked response

01 NORMALIZE REQUEST

Create `DeploymentRequest`; attach policy.

02 SELECT WORKFLOW

Propose a name; application code validates it.

From user request to one checked response

01 NORMALIZE REQUEST

Create `DeploymentRequest`; attach policy.

02 SELECT WORKFLOW

Propose a name; application code validates it.

03 COLLECT EVIDENCE

Collect `CapacityFinding` and `QuotaFinding`.

04 GENERATE CANDIDATE

Return `CandidateRecommendation`.

From user request to one checked response

01 NORMALIZE REQUEST
Create `DeploymentRequest`; attach policy.

02 SELECT WORKFLOW
Propose a name; application code validates it.

03 COLLECT EVIDENCE
Collect `CapacityFinding` and `QuotaFinding`.

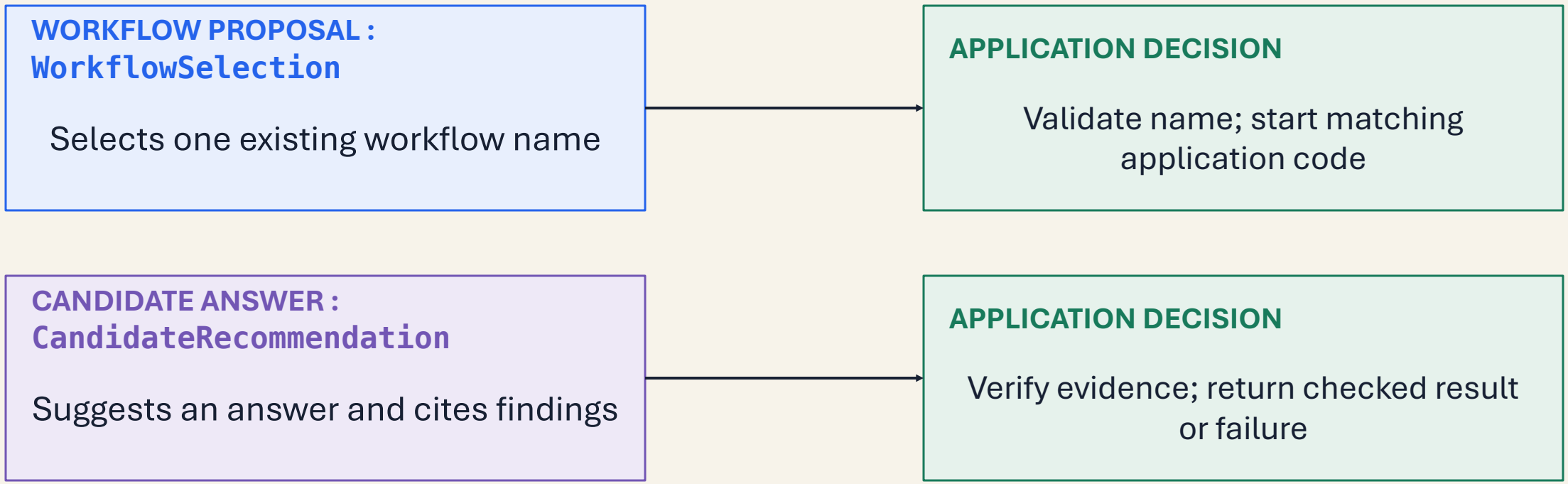
04 GENERATE CANDIDATE
Return `CandidateRecommendation`.

05 VERIFY
Run exact checks; optional LLM critic.

06 RETURN
Return `VerifiedRecommendation` or `ExplicitFailure`.

> Every stage produces an inspectable record or decision; only the controller returns the final response.

Application code validates every model proposal



This fixed workflow has no `ExecutionPlan`: application code already defines the sequence.

Typed workflow proposal

Schema for `WorkflowSelection`

```python
from typing import Literal
from pydantic import BaseModel

WorkflowName = Literal[
    "deploymentRecommendation",
    "quotaExplanation",
]

class WorkflowSelection(BaseModel):
    requestId: str
    selectedWorkflow: WorkflowName
```

WHAT IT MEANS

The Navigator proposes one application-defined workflow.

WHAT IT DOES NOT DO

It is not a URL, tool call, or executed action.

POLICY OWNERSHIP

`DeploymentRequest` keeps the constraints; selection cannot change them.

> Selecting a workflow proposes an implementation; application code still validates the name and starts it.

`CandidateRecommendation` details

Schema for `CandidateRecommendation`

```python
class CandidateRecommendation(BaseModel):
    requestId: str
    answer: str
    capacityFindingId: str
    quotaFindingId: str
```

CREATED BY

A candidate-answer LLM using the request and validated findings.

WHY TWO EXPLICIT FINDING IDS?

The application retrieves and checks each finding and its `requestId` independently.

> A candidate answer is not user input and is not yet a `VerifiedRecommendation`.

Final record details

Checked success response

```
class VerifiedRecommendation(BaseModel):
    requestId: str
    answer: str
    verificationReportId: str
```

Structured failure response

```
class ExplicitFailure(BaseModel):
    requestId: str
    failedCheck: str
    missingEvidence: list[str]
    retryable: bool
```

VerifiedRecommendation
Returned only after every exact requirement passes.

ExplicitFailure
States the failed rule, missing evidence, and retry status.

REQUEST	CAPACITY	QUOTA	CANDIDATE	DECISION
ready	checked	checked	checked	one typed record

An agent task specification defines allowed behavior and application-enforced checks

USER REQUEST	“Recommend one A100 in us-west-2 tonight.”

TASK SCOPE	Produce a recommendation for one A100 deployment; do not provision it.
REQUIRED EVIDENCE	`CapacityFinding` and `QuotaFinding`
ALLOWED TOOLS	`checkCapacity` and `checkQuota`
FORBIDDEN ACTIONS	`forbiddenActions` = [`"provisionGpu"`]
MISSING EVIDENCE POLICY	Stop and return `ExplicitFailure` naming the missing finding.
TERMINAL OUTPUT	`VerifiedRecommendation` or `ExplicitFailure`

> Instructions guide model behavior; application code enforces the evidence, permission, and output rules.

The execution trace explains why verification failed

Events are recorded in time order for `requestId` = "req-599-04".

requestReceived (01)
`DeploymentRequest` recorded

workflowStarted (02)
Deployment workflow started

capacityFindingRecorded (03)
`CapacityFinding` exists

quotaFindingMissing (04)
No `QuotaFinding` recorded

explicitFailureReturned (05)
Required evidence incomplete

AUDIT CONCLUSION

The candidate cannot be accepted because the trace contains no recorded `QuotaFinding` for this request.

> The trace does not decide acceptance. It gives application checks the evidence needed to explain the decision.

SECTION 2

Application-controlled agent runtime architecture

What mechanisms and control methods can an application combine without giving up enforceable policy?



NASA, Apollo 13 Mission Control during the return flight

Four classic agentic mechanisms

These mechanisms describe capabilities used inside a system—not complete production architectures.

TOOL USE

A model proposes a function or tool call.

REFLECTION

A model critiques or revises an intermediate result.

PLANNING

A model proposes future steps or an explicit plan artifact.

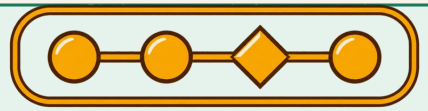
MULTI-AGENT COLLABORATION

Several agents coordinate through routing, delegation, or handoff.

> These four labels describe mechanisms, not complete system architectures.

Three control methods for agentic systems

They differ in whether application code, a step-by-step model loop, or an explicit plan determines the action sequence.



FIXED WORKFLOW

Application code chooses every stage in a predefined sequence.

USE WHEN
Best when required **steps** are **known in advance**.



STEP-BY-STEP AGENT LOOP (ReAct-style)

A model proposes one action after each observation; code validates each proposal.

USE WHEN
Best when the next useful action **depends** on the latest observation.



PLANNER-BASED WORKFLOW

A model proposes a request-specific `ExecutionPlan`; code validates it before execution.

USE WHEN
Best when dependencies or parallel work justify an explicit plan.

> The control method answers: who selects the action sequence, and at what granularity?

Fixed workflow: code chooses every stage

PRO
Easy to test and audit: every step, check, and event is predefined and recorded.

CON
Hard to adapt: unexpected results often require code changes before the next step.

The A100 task fits because its evidence sequence is known in advance.

Step-by-step agent loop (ReAct-style)

The model proposes exactly one next action; application code validates and executes it before the next observation.

1 : AGENT MODEL CALL
Uses current context. Proposes one allowed action.

→

2 : CONTROLLER CHECK
Accepts one allowed proposal or rejects it before execution.

→

3 : IMPLEMENTATION
Runs one approved tool, function, or specialist call.

→

4 : OBSERVATION
Records the result and supplies it to the next model call.

LOOP: the recorded observation becomes input to the next model call.

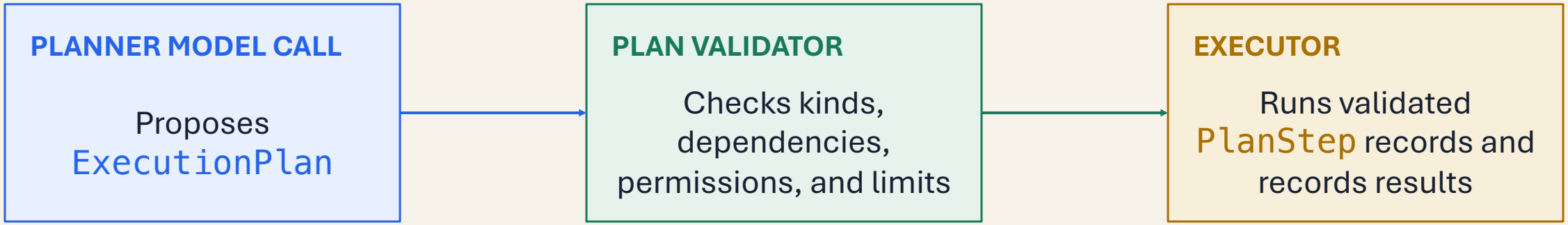
PRO
Adapts after each observation.

CON
Requires proposal checks, stop limits, and final verification.

ReAct is one well-known implementation of this broader step-by-step control method.

Planner workflow

A planner model call proposes an `ExecutionPlan` containing several `PlanStep` records and their dependencies.



PRO	Makes dependencies, parallel work, and plan-level checks explicit.
CON	Adds planning cost plus validation, execution, and replanning logic.

> Use planning when an explicit dependency graph improves control.

SECTION 4

Choosing a control method from task structure

Which task properties favor a fixed workflow, step-by-step loop, or planner-based workflow?

Which control method fits the A100 recommendation task?

Choose before the reveal. Base the choice on task structure, not on how many agents might participate.

A100 TASK FACTS

The required stages are known before execution. Capacity and quota evidence are both mandatory. The candidate must cite both findings. `provisionGpu` is forbidden. A missing record returns `ExplicitFailure`.

A : FIXED WORKFLOW

Application code defines the required sequence.

B : STEP-BY-STEP LOOP

A model chooses one next action after each observation.

C : PLANNER-BASED

A model proposes an explicit dependency graph.

Vote: A, B, or C. Be ready to name the deciding task property.

A fixed workflow fits the A100 recommendation task

The required evidence, order, rejection behavior, and terminal record are known before execution.

1 : VALIDATE REQUEST	Create `DeploymentRequest` and attach policy.
2 : COLLECT FINDINGS	Require `CapacityFinding` and `QuotaFinding`.
3 : GENERATE CANDIDATE	Create `CandidateRecommendation` with both IDs.
4 : VERIFY + RETURN	Return `VerifiedRecommendation` or `ExplicitFailure`.

DECIDING EVIDENCE No unexpected observation or dependency graph changes the next stage.

Step-by-step control for a single-agent FAQ lookup

Example: to answer “Can I change my plan without a fee?”, the assistant checks account facts, searches approved policy text, and decides whether another lookup is needed.

1 : FAQ MODEL CALL

Uses current context. Proposes one allowed lookup.

→

2 : CONTROLLER CHECK

Validates the proposal and applies step and cost limits.

→

3 : APPROVED TOOL

`lookupAccount` returns plan/status.
`searchPolicy` returns the rule.

→

4 : OBSERVATION

The typed tool result becomes input to the next model call.

LOOP: repeat only when the new observation makes another lookup useful.

WHY ONE AGENT FITS

One user question, shared context, and two read-only retrieval tools.

This is a read-only customer-support example: the agent retrieves facts and policy, but cannot change the account.

Task structure selects the control method

FIXED WORKFLOW A100 recommendation	**WHY THIS METHOD FITS** Collect two known findings, generate a candidate, verify, and return.
STEP-BY-STEP LOOP FAQ lookup	**WHY THIS METHOD FITS** Choose `lookupAccount`, `searchPolicy`, or stop after each observation.
PLANNER-BASED WORKFLOW Python test diagnosis	**WHY THIS METHOD FITS** Read the test output and dependency changes; inspect affected code; combine findings into a diagnosis.

> Control method selects the action sequence; functions, tools, model calls, or specialist agents perform the work.

SECTION 6

Planner-based workflows

How can a model propose a request-specific plan while application code controls execution and acceptance?

TECHNICAL MAPPING

A plan is a dependency graph: it exposes ordering, parallel work, and the checks that must precede execution.

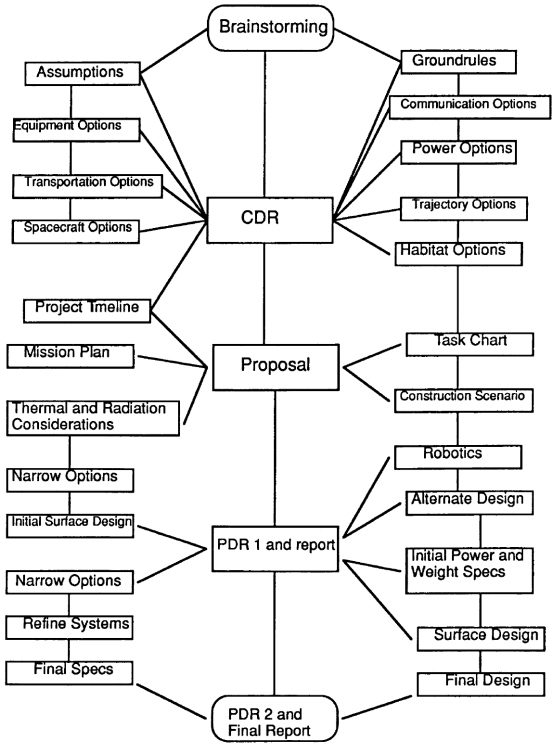


Figure 11.5. PERT/CPM Chart.

86

A task-dependency chart shows which activities can run in parallel and which must wait for earlier work.
University of Texas at Austin / NASA NTRS, FLARE report

Fixed and planner-based workflows fit different dependency structures

FIXED WORKFLOW : A100 RECOMMENDATION

Known sequence

1. Validate request
2. Collect capacity and quota findings
3. Generate candidate
4. Verify and return

No `ExecutionPlan` is created.

PLANNER-BASED : PYTHON TEST DIAGNOSIS

Request-specific dependencies

Read test output and dependency changes independently. Then inspect affected code. Finally combine findings into a diagnosis.

`ExecutionPlan` makes dependencies explicit.

> Use planning when an explicit dependency graph improves control, parallelism, or validation.

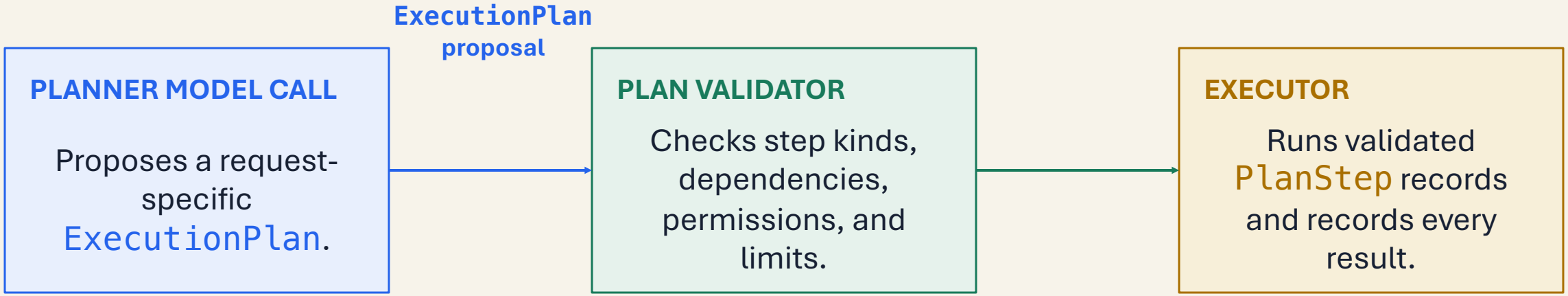
Planner proposes an `ExecutionPlan`

PLANNER MODEL CALL

Proposes a request-specific `ExecutionPlan`.

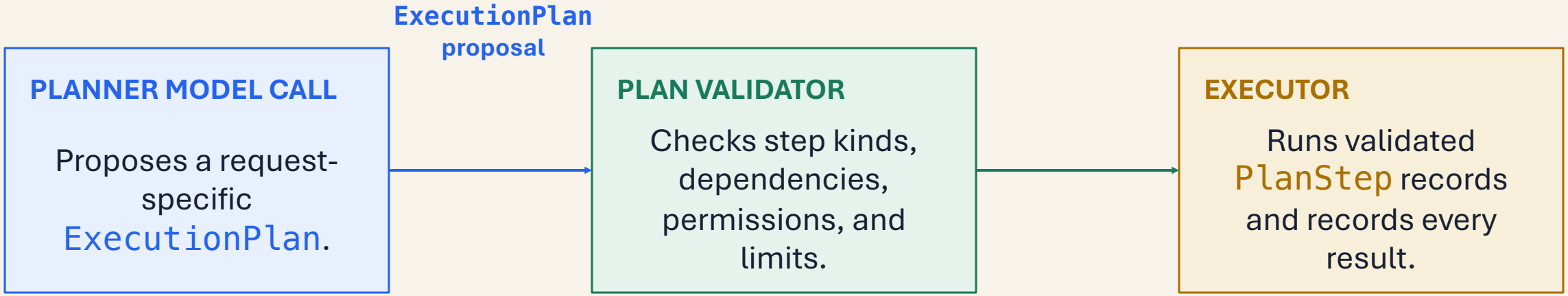
> `ExecutionPlan` is a model proposal—not a tool call and not executed code.

Application code validates before execution



The validator accepts or rejects the complete plan before any step runs.

Replanning changes steps, not permissions



Replanning can revise proposed work but cannot revise application policy.

Planner example: diagnose a Python test failure

USER REQUEST

“Find why this Python test started failing after a dependency update. Do not change the code.”

s1 : readTestOutput
Read the failing test output.

s2 : inspectDependencyChanges
Inspect the updated dependency.

s3 : inspectAffectedCode
Inspect code identified by the error; depends on s1.

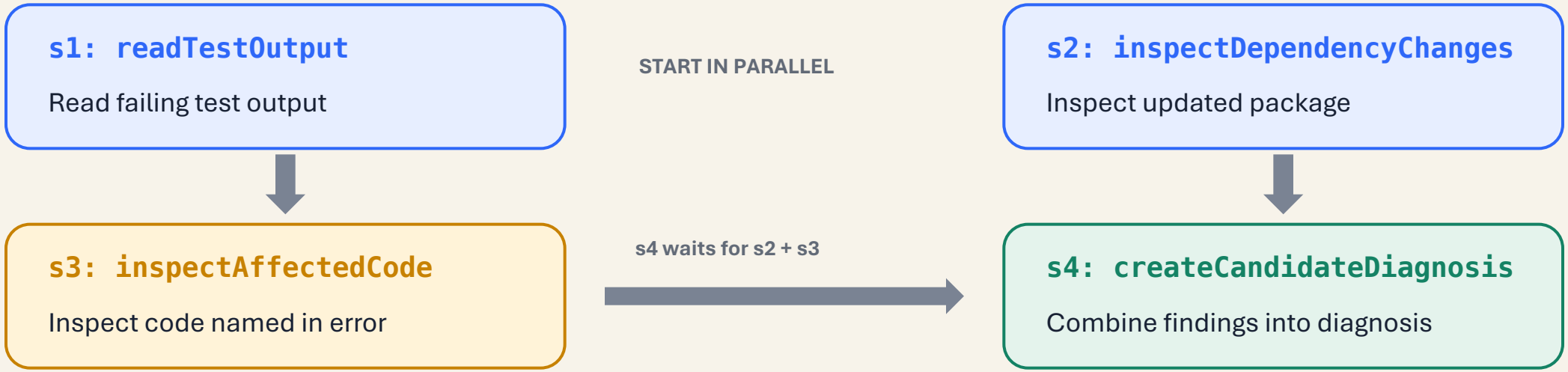
s4 : createCandidateDiagnosis
Combine dependency and code findings; depends on s2 and s3.

> An explicit plan helps because two checks can start independently while later steps have clear dependencies.

Planner example: diagnose a Python test failure

USER REQUEST

"Find why this Python test started failing after a dependency update. Do not change the code."



> An explicit plan helps because two checks can start independently while later steps have clear dependencies.

Planner inputs constrain the plan proposal

APPLICATION-SUPPLIED INPUT

Diagnose a Python test failure after a dependency update. Do not change code.

Allowed:

- `readTestOutput`: `inspectDependencyChanges`
- `inspectAffectedCode`: `createCandidateDiagnosis`

Forbidden: `writeFile`: `applyPatch`

PLANNER OUTPUT

`ExecutionPlan`

Contains ordered or dependent `PlanStep` records.

Each step names an allowed operation, arguments, dependencies, and expected output.

It does not grant new permissions.

> The planner chooses among allowed read-only steps; application policy defines the allowed set.

Typed `PlanStep` and `ExecutionPlan` proposals

Schemas for `PlanStep` and `ExecutionPlan`

```
StepKind = Literal[
    "readTestOutput",
    "inspectDependencyChanges",
    "inspectAffectedCode",
    "createCandidateDiagnosis",
]
class PlanStep(BaseModel):
    stepId: str
    kind: StepKind
    arguments: dict
    dependsOn: list[str]
    expectedOutput: str

class ExecutionPlan(BaseModel):
    requestId: str
    steps: list[PlanStep]
```

`PlanStep`
One operation, its arguments, dependencies, and expected record.

`ExecutionPlan`
The request-specific proposal containing every `PlanStep` record.

VALIDATION
Application code validates the plan before any step can run.

Exact rules reject invalid plans

STEP KIND

Every step kind is in the closed read-only set.

ARGUMENTS

Arguments match the schema for that step kind.

DEPENDENCIES

Every `dependsOn` reference exists and the graph is acyclic.

PERMISSIONS

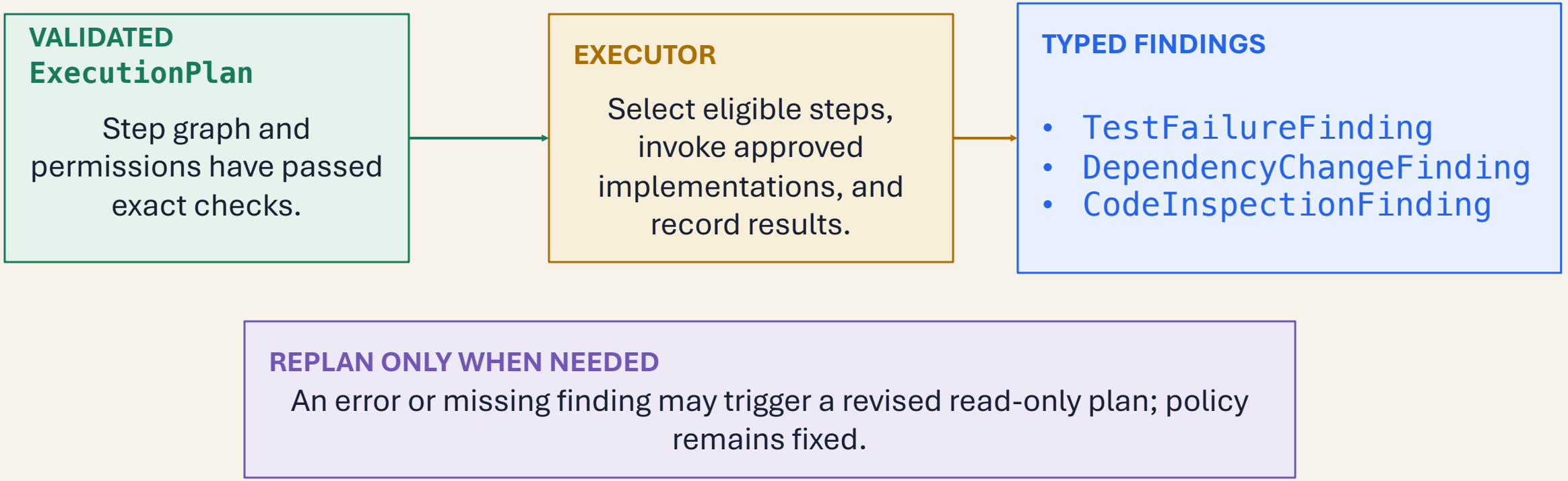
No step requests `writeFile` or `applyPatch`.

LIMITS

Step count, replans, time, and cost stay within application limits.

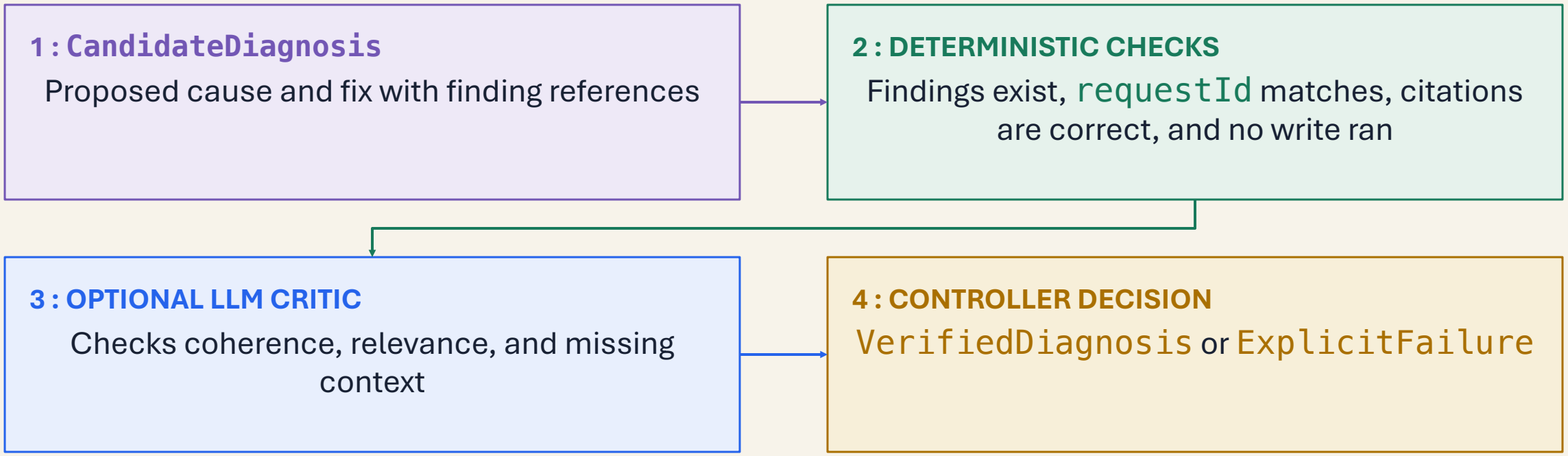
> Plan validation checks the complete `ExecutionPlan`; it does not rely on the planner to obey its own prompt.

The executor runs only validated `PlanStep` records



Execution and replanning remain application-controlled even when the plan was model-generated.

Final verification remains separate from planning



An LLM critic is probabilistic; it cannot replace exact application checks.

> Verification may include an agent; the final acceptance gate remains application-controlled.

Lab: implement and test the controlled A100 workflow

01 Define `DeploymentRequest`, `WorkflowSelection`, both finding records, and both terminal response types.

02 Implement Navigator-to-workflow selection from the closed `WORKFLOWS` map.

03 Implement `checkCapacity` and `checkQuota` as fixed fake read-only dependencies.

04 Reject `provisionGpu` proposals and require both findings before synthesis.

05 Record `ExecutionTrace` events and return `VerifiedRecommendation` or `ExplicitFailure`.

Conceptual test assertions

```
assert workflowAllowed(selection)
assert provisioningForbidden(request)
assert bothFindingsPresent(candidate)
assert requestIdsMatch(finalResponse)
assert terminalType(finalResponse)
```

> The base lab implements the fixed workflow; a planner-based variant is a separate extension.

Appendix: `WorkflowSelection`

Schema and controller lookup for `WorkflowSelection`

```
WorkflowName = Literal[
    "deploymentRecommendation",
    "quotaExplanation",
]

class WorkflowSelection(BaseModel):
    requestId: str
    selectedWorkflow: WorkflowName

def chooseWorkflow(request):
    selection = callNavigator(
        request, outputType=WorkflowSelection
    )
    requireSameRequest(
        request.requestId, selection.requestId
    )
    return WORKFLOWS[selection.selectedWorkflow]
```

Appendix: Python-test execution plan schema

Schemas for `PlanStep` and `ExecutionPlan`

```
StepKind = Literal[
    "readTestOutput", "inspectDependencyChanges",
    "inspectAffectedCode", "createCandidateDiagnosis",
]

class PlanStep(BaseModel):
    stepId: str
    kind: StepKind
    arguments: dict
    dependsOn: list[str]
    expectedOutput: str

class ExecutionPlan(BaseModel):
    requestId: str
    steps: list[PlanStep]
```

Appendix: validate plans before execution

Application code validates the complete `ExecutionPlan`

```
def validatePlan(request, plan):
    requireSameRequest(request.requestId, plan.requestId)
    requireUniqueStepIds(plan.steps)
    requireAcyclicDependencies(plan.steps)

    allowed = {
        "readTestOutput", "inspectDependencyChanges",
        "inspectAffectedCode", "createCandidateDiagnosis",
    }
    for step in plan.steps:
        require(step.kind in allowed)
        requireValidArguments(step)
        require(step.kind not in request.forbiddenActions)

    return plan
```

Appendix: fixed A100 workflow in Python

Application code for the fixed workflow

```
def runDeploymentRecommendation(request):
    capacity = checkCapacity(request)
    quota = checkQuota(request)
    candidate = callRecommendationModel(
        request, capacity, quota
    )
    report = verifyRecommendation(
        request, candidate, capacity, quota
    )

    if not report.passed:
        return ExplicitFailure.fromReport(report)

    return VerifiedRecommendation.fromCandidate(
        candidate, report
    )
```

Appendix: complete request trace

ExecutionTrace for req-599-04

```
req-599-04  requestReceived        DeploymentRequest
req-599-04  requestValidated       policy attached
req-599-04  workflowSelected       deploymentRecommendation
req-599-04  workflowStarted        fixed workflow
req-599-04  toolCalled             checkCapacity
req-599-04  findingRecorded        cap-usw2-0913
req-599-04  toolCalled             checkQuota
req-599-04  findingRecorded        quota-ee-0914
req-599-04  candidateCreated       CandidateRecommendation
req-599-04  verificationPassed     VerificationReport
req-599-04  finalResponseReturned  VerifiedRecommendation
```

Appendix: definitions for emerging runtime features

COMPACTION	Select prior state and rewrite it into a smaller context for a later model call.
BACKGROUND EXECUTION	Continue a long-running task without keeping one foreground request open.
MID-TURN STEERING	Let new user input influence a run before the current work finishes.
DYNAMIC TOOL DISCOVERY	Load tool descriptions only when the current stage needs them.
SUBAGENT TREE	Create temporary child agents under one root controller for specific subtasks.

Deferred: asyncio scheduling, cancellation, timeouts, and failure propagation.

Sources: developers.openai.com/api/docs/guides/compaction and /tools/tool-search

Appendix: source roles and maturity

SOURCE FAMILY	LECTURE ROLE	MATURITY
ReAct, Andrew Ng, Building Effective Agents	Historical vocabulary	2022–2024
Context engineering, containment, evals	General design guidance	Current research
OpenAI platform, tracing, tool search, compaction	Provider examples	Current documentation
Google scaling, Anthropic multi-agent	Coordination evidence	Published studies
MCP and A2A communication protocols	Related protocols	Current specifications

Primary sources: research.google, developers.openai.com, anthropic.com/engineering, modelcontextprotocol.io, a2a-protocol.org